Python Programming Handbook

Beginner Friendly Learning Guide

Advanced Python 1

Following are some of the newly added features in Python programming language.

Walrus Operator

The walrus operator (:=), introduced in Python 3.8, allows you to assign values to variables as part of an expression.

```python
# Using Walrus operator
if (n := len([1, 2, 3, 4, 5])) > 3:
    print(f"List is too long ({n} elements, expected <= 3)")
```

In this example, n is assigned the value of len([1, 2, 3, 4, 5]) and then used in the comparison within the if statement.

Types Definitions In Python

Type hints are added using the colon (:) syntax for variables and the → syntax for function return types.

```python
# Variable type hint
age: int = 25

# Function type hints
def greeting(name: str) -> str:
    return f"Hello, {name}!"

# Usage
print(greeting("Alice"))
```

Advanced Type Hints

Python's typing module provides more advanced type hints, such as List, Tuple, Dict and Union.

```python
from typing import List, Tuple, Dict, Union
```

```python
from typing import List, Tuple, Dict, Union

# List of integers
numbers: List[int] = [1, 2, 3, 4, 5]

# Tuple of a string and an integer
person: Tuple[str, int] = ("Alice", 30)

# Dictionary with string keys and integer values
scores: Dict[str, int] = {"Alice": 90, "Bob": 85}

# Union type
identifier: Union[int, str] = "ID123"
```

Match Case

Python 3.10 introduced the match statement, which is similar to the switch statement found in other programming languages.

```
def http_status(status):
    match status:
        case 200:
            return "OK"
        case 404:
            return "Not Found"
        case 500:
            return "Internal Server Error"
        case _:
            return "Unknown status"

print(http_status(200))
print(http_status(404))
```

Dictionary Merge & Update Operators

New operators | and |= allow for merging and updating dictionaries.

```
dict1 = {'a': 1, 'b': 2}
dict2 = {'b': 3, 'c': 4}

merged = dict1 | dict2

print(merged)
```

```
with (
    open('file1.txt') as f1,
    open('file2.txt') as f2
):
    # Process files
    pass
```

Exception Handling In Python

There are many built-in exceptions which are raised in python when something goes wrong.

```
try:
    # Code which might throw exception
    print(10 / 0)

except Exception as e:
    print(e)
```

```
try:
    # Code

except ZeroDivisionError:
    # Code

except TypeError:
    # Code

except:
    # All other exceptions
```

Raising Exceptions

We can raise custom exceptions using the raise keyword in python.

```
age = -5

if age < 0:
    raise ValueError("Age cannot be negative")
```

Try With Else Clause

```
try:
    # Some code

except:
    # Some code

else:
    # Executed if try was successful
```

Try With Finally

```
try:
    # Some code

except:
    # Some code

finally:
    # Executed regardless of error
```

If __name__ == '__main__' In Python

'__name__' evaluates to the name of the module in python from where the program is ran.

If the module is being run directly from the command line, the '__name__' is set to string "__main__".

Thus, this behaviour is used to check whether the module is run directly or imported to another file.

```
if __name__ == "__main__":
    print("Running directly")
```

The Global Keyword

'global' keyword is used to modify the variable outside of the current scope.

```
x = 10

def change():
    global x
    x = 20
```

Enumerate Function In Python

The 'enumerate' function adds counter to an iterable and returns it.

```
list1 = ["Harry", "Rohan", "Shubham"]

for i, item in enumerate(list1):
    print(i, item)
```

List Comprehensions

List Comprehension is an elegant way to create lists based on existing lists.

```
list1 = [1,7,12,11,22]

list2 = [item for item in list1 if item > 8]
```

Advanced Python 2

Virtual Environment

An environment which is same as the system interpreter but is isolated from the other Python environments on the system.

Installation

To use virtual environments, we write:

```
pip install virtualenv
```

We create a new environment using:

```
virtualenv myprojectenv
```

The next step after creating the virtual environment is to activate it.

We can now use this virtual environment as a separate Python installation.

Pip Freeze Command

'pip freeze' returns all the package installed in a given python environment along with the versions.

```
pip freeze > requirements.txt
```

The above command creates a file named 'requirements.txt' in the same directory containing the output of 'pip freeze'.

We can distribute this file to other users, and they can recreate the same environment using:

```
pip install -r requirements.txt
```

Lambda Functions

Function created using an expression using 'lambda' keyword.

Syntax:

```
lambda arguments: expression
```

Example:

```
square = lambda x:x*x

square(6)
sum = lambda a,b,c:a+b+c
sum(1,2,3)
```

Join Method (Strings)

Creates a string from iterable objects.

```
l = ["apple", "mango", "banana"]

result = ", and, ".join(l)

print(result)
```

The above line will return: “apple,and,mango,and,banana”.

Format Method (Strings)

Formats the values inside the string into a desired output.

```
template.format(p1,p2...)
```

Syntax:

```
"{} is a good {}".format("harry", "boy")

"{1} is a good {0}".format("harry", "boy")
```

Map, Filter & Reduce

Map applies a function to all the items in an input_list.

Syntax:

```
map(function, input_list)
```

Filter creates a list of items for which the function returns true.

```
list(filter(function))
```

Reduce applies a rolling computation to sequential pair of elements.

```
from functools import reduce

val = reduce(function, list1)
```

If the function computes sum of two numbers and the list is [1,2,3,4]